

4주차 예습 과제

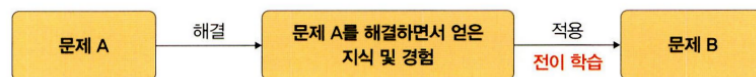
📖 딥러닝 파이토치 교과서 5.3 - 5.5 장

https://colab.research.google.com/drive/1fk4C1rSvssmKoR_k7TxGQ0U6lcUpi_q4#scrollTo=9GVwOuDJVgeg

5.3 전이 학습

- 전이 학습(transfer learning)
 - 이미지넷ImageNet 처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보장해서 사용하는 것
 - 아주 큰 데이터셋을 사용하여 훈련된 모델을 사전 훈련된 모델(네트워크)이라고 함
 - 비교적 적은 수의 데이터를 가지고도 우리가 원하는 과제를 해결할 수 있음

▼ 그림 5-29 전이 학습

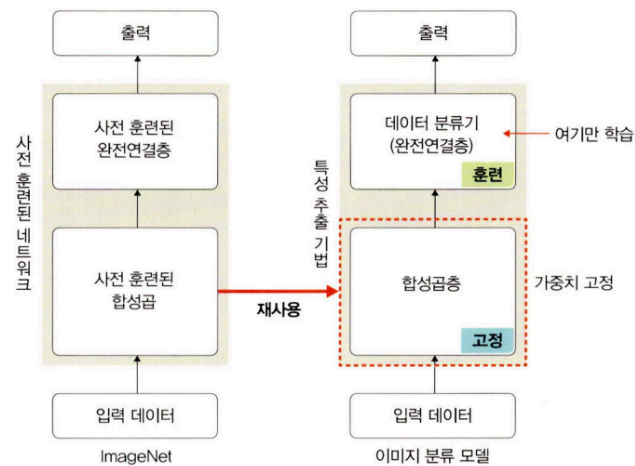


- 특성 추출과 미세 조정 기법이 있음

5.3.1 특성 추출 기법

- 특성 추출(feature extractor)은 ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만들
- 학습할 때는 마지막 완전연결층(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층들은 학습되지 않도록 함
- 특성 추출은 이미지 분류를 위해 두 부분으로 구성됨
 - **합성곱층**: 합성곱층과 풀링층으로 구성
 - **데이터 분류기(완전연결층)**: 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분
- 사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시키고, 그 출력을 데이터 분류기에서 훈련
- 사용 가능한 이미지 분류 모델
 - Xception
 - Inception V3
 - ResNest50
 - VGG16
 - VGG19
 - MobileNet

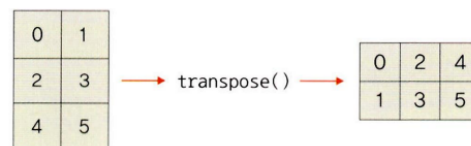
▼ 그림 5-30 특성 추출 기법



▼ `np.transpose` (코드 5-14)

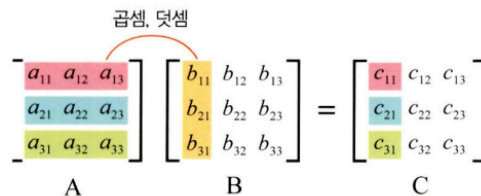
- 행과 열을 바꿈으로써 행렬의 차원을 바꾸어 주는 코드

▼ 그림 5-33 `transpose()`



- 일반적으로 행렬의 차원을 변경하는 이유는 행렬의 내적 연산 때문
- 다음 그림과 같이 A라는 행렬과 B라는 행렬의 내적 결과는 C가 됨

▼ 그림 5-34 행렬의 내적



$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

- 이때 A 행렬의 행과 B 행렬의 열의 수가 같아야 내적이 가능
- A와 B의 행과 열의 수가 달라 내적이 불가능할 때는 `np.transpose()` 나 `np.reshape()` 등으로 차원을 조정해야 함

▼ ResNet18

- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공
- 입력 제약이 매우 크고, 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있음

▼ 사전 훈련된 모델

- 파이토치는 다음과 같은 방법으로 무작위의 가중치로 모델을 구성할 수 있음

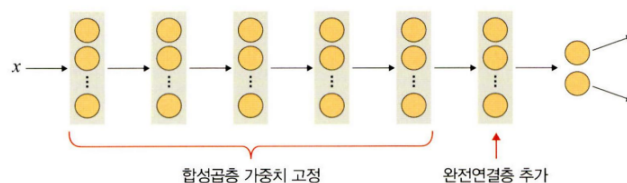
```
import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

- `pretrained=True` 로 설정하여 사용하면 사전 학습된 모델의 가중치 값 사용할 수 있음

▼ ResNet18의 마지막 부분에 완전연결층을 추가 (코드 5-17)

- 추가된 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용됨

▼ 그림 5-36 ResNet18에 완전연결층 추가



▼ `tensor.clone()`, `tensor.detach()`, `tensor.clone().detach()` 비교 (코드 5-28)

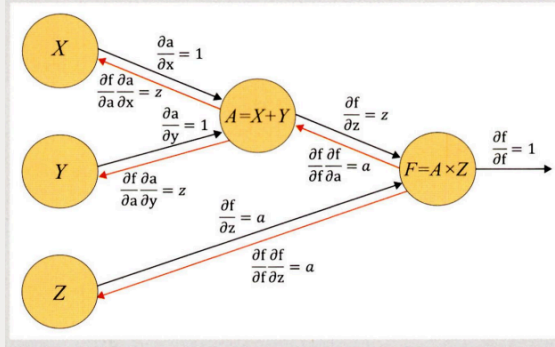
▼ 표 5-2 `tensor.clone()`, `tensor.detach()`, `tensor.clone().detach()`의 비교

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

▼ 계산 그래프(computational graph)

- 계산 과정을 그래프로 나타낸 것
- 여러 개의 노드(node)와 그 노드들을 연결하는 선인 에지(edge)로 구성

♥ 그림 5-39 계산 그래프

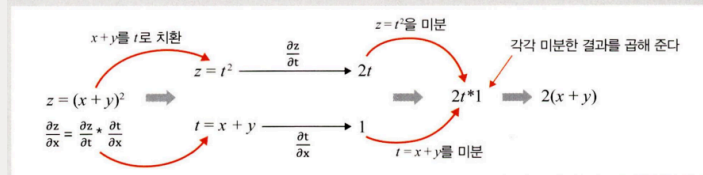


- 계산 그래프를 사용하는 이유
 - 국소적 계산 가능
 - 그림에서 Z값이 변경되었다면 X, Y 계산 결과를 그대로 유지한 채로 바뀐 Z의 연산이 필요한 $F = A \times Z$ 만 계산하면 됨
 - 역전파를 통한 미분 계산이 편리
 - 그림의 주황색 선이 역전파를 구하는 과정을 보여주는데, 연쇄 법칙(chain rule)을 이용하여 빠르고 간편하게 미분을 계산할 수 있음

▼ 연쇄 법칙

- 두 개 이상의 함수가 결합된 함수, 즉 합성 함수의 미분법을 연쇄 법칙 혹은 체인룰(chain rule)이라고 함
- 다음 그림과 같이 $z = (x + y)^2$ 을 $z = t^2$ 과 $t = x + y$ 의 합성 함수로 표현하고 각 합성 함수의 미분 결과를 곱한 결과로 나타내는 것

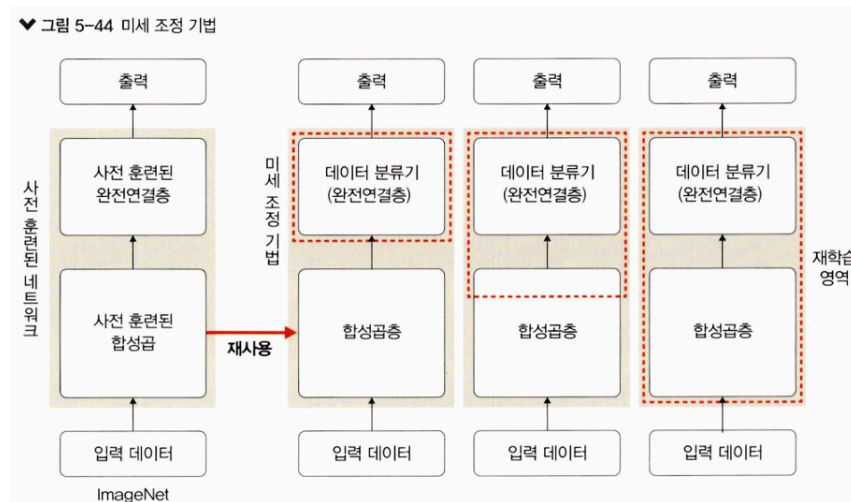
♥ 그림 5-40 합성 함수의 미분



5.3.2 미세 조정 기법

- 미세 조정(fine-tuning) 기법은 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식
 - 특성 추출은 목표 특성을 잘 추출했다는 전제 하에 좋은 성능을 낼 수 있음
 - 특성이 잘못 추출되었다면 미세 조정 기법으로 새로운 이미지 데이터를 사용하여 네트워크의 가중치를 업데이트해서 특성을 다시 추출할 수 있음
 - 즉, 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것
- 미세 조정 기법은 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정하는 기법
 - 많은 연산량이 필요하기 때문에 GPU 사용을 권장
- 훈련시키려는 데이터셋의 크기와 사전 훈련된 모델에 따라 다음 전략 세울 수 있음
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우: 모델 전체를 재학습. 데이터셋 크기가 크기 때문에 재학습시키는 것이 좋은 전략.

- **데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우:** 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기를 학습. 데이터셋이 유사하기 때문에 전체를 학습시키는 것보다는 강한 특징이 나타나는 합성곱층의 뒷부분과 데이터 분류기만 새로 학습해도 최적의 성능 낼 수 있음.
- **데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우:** 합성곱층의 일부분과 데이터 분류기를 학습. 데이터가 적기 때문에 일부 계층에 미세 조정 기법을 적용한다고 해도 효과가 없을 수 있음. 따라서 합성곱층 중 어디까지 새로 학습 시켜야 할지 적당히 설정해주어야 함.
- **데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우:** 데이터 분류기만 학습. 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 과적합이 발생할 수 있음. 따라서 최종 데이터 분류기인 완전연결층에 대해서만 미세 조정 기법을 적용.

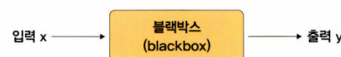


- 미세 조정은 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주게 되면 과적합 문제가 발생할 수 있기 때문에 정교하고 미세한 파라미터 업데이트가 필요

5.4 설명 가능한 CNN

- 설명 가능한 CNN(explainable CNN)은 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술
- CNN은 블랙박스과 같아 내부에서 어떻게 동작하는지 설명하기 어려움

▼ 그림 5-45 CNN의 블랙박스



- 따라서 CNN으로 얻은 결과는 신뢰하기 어려운데, 이를 해결하려면 CNN 처리 과정을 시각화해야 할 필요성이 있음
- CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 특징이 어떻게 추출되고 학습하는지를 시각적으로 설명할 수 있어야 결과에 대한 신뢰성을 얻을 수 있음
- CNN의 시각화 방법에는 필터에 대한 시각화와 특성맵에 대한 시각화가 있음

5.4.1 특성 맵 시각화

- 특성 맵(혹은 활성화 맵)은 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
- 특정 입력 이미지에 대한 특성 맵을 시각화한다는 의미는 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것
- ▼ 소프트맥스와 로그 소프트맥스 수식

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\begin{aligned} \text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right) \end{aligned}$$

- CN은 필터와 특성 맵을 시각화해서 CNN 결과의 신뢰성을 확보할 수 있음

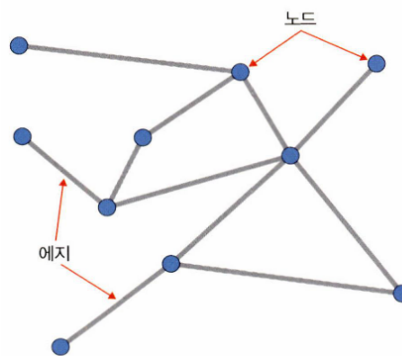
5.5 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(graph convolutional network)는 그래프 데이터를 위한 신경망

5.5.1 그래프란

- 그래프는 방향성이 있거나(directed) 없는(undirected) 에지로 연결된 노드(nodes=vertices)의 집합
- 여기에서 노드와 에지는 일반적으로 풀고자 하는 문제에 대한 전문가 지식이나 직관 등으로 구성됨

▼ 그림 5-50 그래프



- 노드(node, vertex): 파란색 원 → 원소들을 의미
- 에지(edge): 두 노드를 연결한 선 → 결합 방법을 의미

5.5.2 그래프 신경망

- 그래프 신경망(Graph Neural Network, GNN)은 그래프 구조에서 사용하는 신경망을 의미
- 그래프 데이터에 대한 표현은 다음과 같이 두 단계로 이루어짐

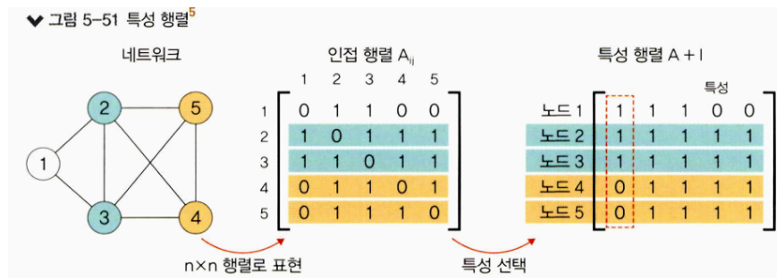
1단계. 인접 행렬(adjacency matrix)

- 네트워크가 있을 때 노드 n개를 nxn 행렬로 표현
- 이렇게 생성된 인접 행렬 내의 값은 ' A_{ij} 는 i와 j의 관련성 여부'를 만족하는 값으로 채워짐

즉, 인접 행렬 과정은 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정

2단계. 특성 행렬(feature matrix)

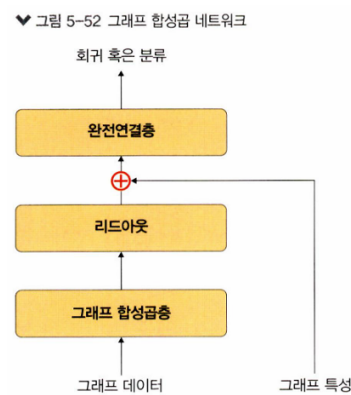
- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용
- 각 입력 데이터에서 이용할 특성을 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미(Ex- 첫 번째 행은 첫 번째 노드의 특성 값)
- 이제 노드 1·2·3·4에 대한 특성을 한눈에 파악하기 쉽게 표현됨



즉, 특성 행렬 과정을 거쳐 그래프 특성이 추출됨

5.5.3 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(Graph Convolutional Network, GCN)는 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘
- 그래프 합성곱 네트워크 구조는 다음 그림과 같음



- 리드아웃(readout)은 특성 행렬을 하나의 벡터로 변환하는 함수
 - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성
- GCN에서 가장 중요한 부분은 그래프 합성곱층(graph convolutional layer)
 - 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있기 때문(기존 그래프 형태의 데이터로는 딥러닝 알고리즘을 적용할 수 없음)
- GCN은 다음과 같은 곳에서 활용
 - SNS에서 관계 네트워크
 - 학술 연구에서 인용 네트워크
 - 3D Mesh